Android - Unit 6

Data Persistence

User Interfaces 2

Unit 6: Data Persistence

- ~~Pathway 1 Introduction to SQL~~
- ~~Pathway 2 Use Room for Data Persistence~~
  - Activity 1 Video: Kotlin flows in Practice
- Pathway 3: Store and access data using keys with DataStore

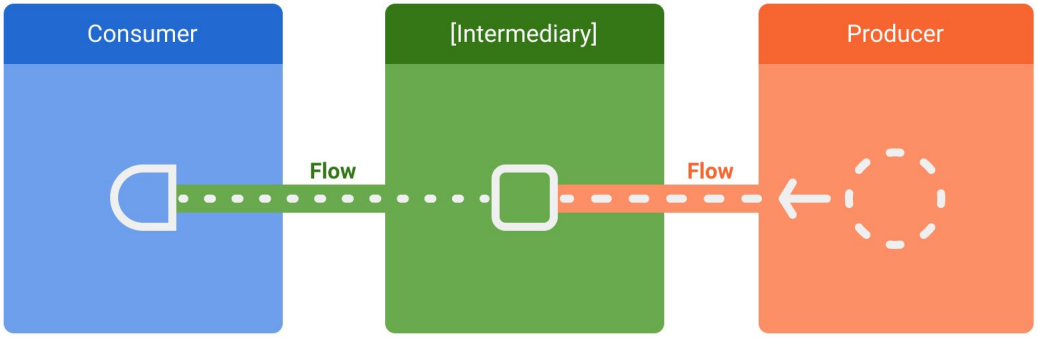
PW2 Video Kotlin Flows

[Kotlin flows on Android]



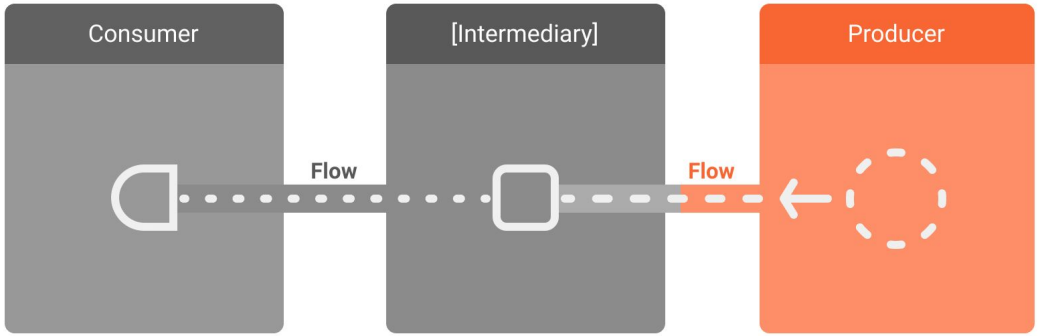
Flow overzicht

- Kotlin Flows lijken op Java Streams, maar gebruiken coroutines en kunnen asynchroon zijn



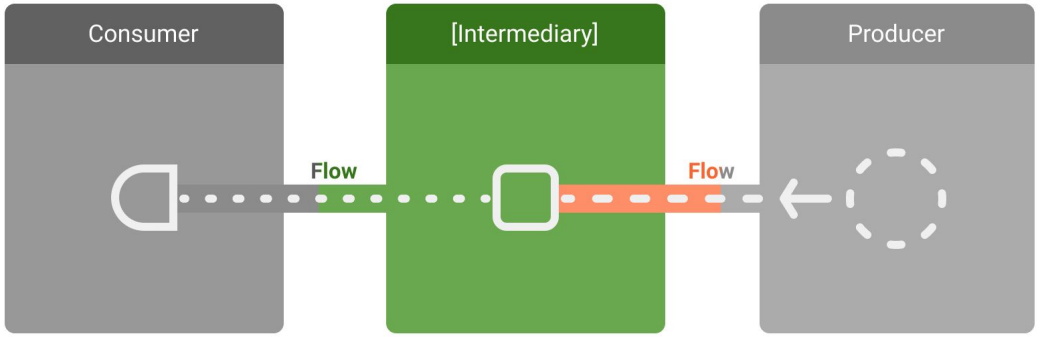
Flow: Producer

- Flow builder
  - `flow {...}`: `Flow<T>` coroutine
    - `emit(data:T)` in coroutine blok
- Veel frameworks geven je al een Flow terug (Room, DataStore...)
- Gelijkaardig aan Java Collections stream() en dotnet LINQ



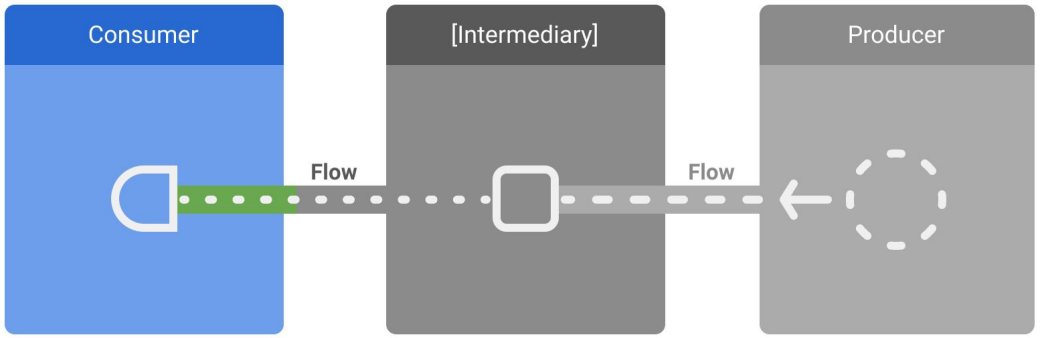
Flow: Intermediair

- “Intermediary Operator”
- Passen de data flow aan (operatie op elk element in de pipe)
  - `map<T,R>{...}`
  - `catch(Throwable t)`: werkt als een try catch rond alle upstream operaties
  - `filter()...`
- zijn geen suspend functies



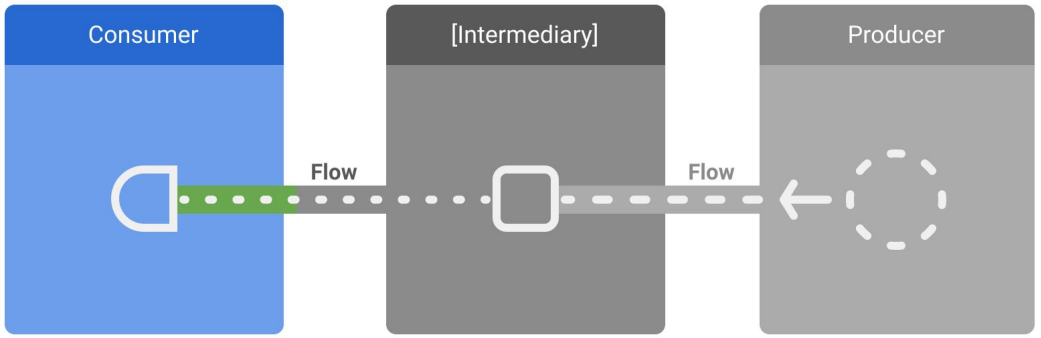
Flow: Consumer

- Consumers zijn suspend functies
  - `.collect {...}`
    - "Terminal operator"
    - `{...}` lambda wordt aangeroepen voor **elke** element
    - Pullt data doorheen de flow
  - `.reduce()...`
- `.value()` kijkt **synchroon** naar laatst gepubliceerde staat (geen echte consumer, pullt geen data)



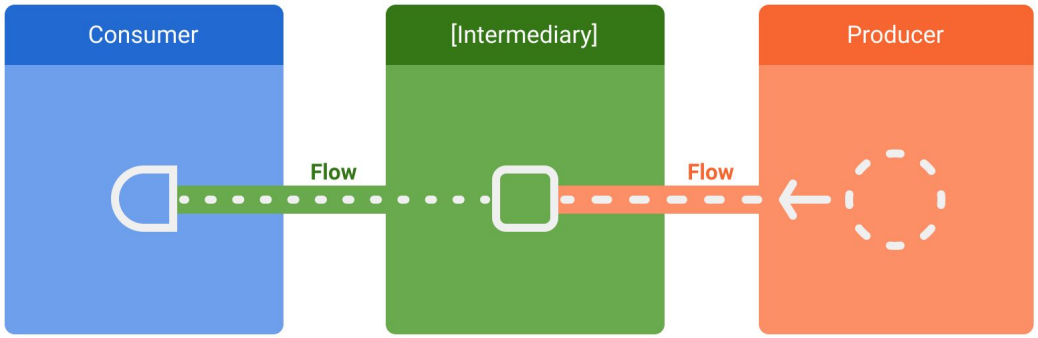
Flow: Consumer

- Standaard zijn flows
  - *Lazy*: data productie wordt geïnitieerd door consumer (e.g. "`.collect()`")
  - *Cold*: herstart data-collection vanaf het begin voor elke nieuwe consumer
    - bij een hot flow leest een collect() emitted data uit memory en triggert geen rerun van de Producer



Flow in Android Compose

- We gebruikten `StateFlow` eerder in `ViewModel`
  - kanLifecycle gevoelig zijn
- Waarom gebruikten we voor [Mutable]StateFlow geen coroutines?
  - de synchrone collectAsState[withLifecycle] compose extension function start intern een coroutine. Elke keer een nieuwe value geëmit wordt, update de state en triggert een recompositie van de UI



PW3 Store and access keys with DataStore



PW3 Store and access keys with DataStore



- Room: OO communicatie met lokale android RDBMS
- DataStore: kleine datasets, geen partial updates
  - Preferences: name value pairs
  - Proto: typed objects

PW3 Store and access keys with DataStore



- Lab: Save **Preferences** locally with DataStore
  - Gebruikt flows
    - stateIn() intermediaire operatie om de cold flow hot te maken: geeft laatste waarde aan alle consumers
  - Gebruikt manual dependency injection
    - A sample application with Koin
- Practice: Create a flight search app
- Badge!



Codelab dessert datastore voorbeeld met Koin:
https://gitlab.com/kdg-ti/user-interfaces-2/android/examples/u06_persistence/dessertreleases_koin
(newer koin compiler plugin variant in koin-compiler-plugin branch)